Using Bodhan AI's Indic ASR in Production

What we found after independently testing indic-transcribe-core and indic-transcribe-flex — and what to do about it

2026-09-24

Who this is for

You're deciding whether to use **Bodhan AI's** `indic-transcribe-core` (or its sibling `indic-transcribe-flex`) for a real product — call center transcripts, voice notes, IVR, meeting notes, customer support in Indian languages. This model is genuinely strong: on our tests it transcribes clean audio in Hindi, Tamil, Telugu, Kannada, Malayalam, Urdu, Bhojpuri, and Maithili at roughly 80–85% word accuracy. That's usable.

This guide is not that pitch. It's the five things that will bite you in production if you don't know about them going in, each with a number attached and a fix. Everything here comes from an independent test suite we built and ran ourselves — not vendor marketing, not the model card alone. Full technical report with every number, confidence interval, and reproducible script: `REPORT.md` in this repository.

One-paragraph summary of the model itself: it's a 1.2B-parameter speech model, built on NVIDIA's Canary architecture, released by Bodhan AI in partnership with AI4Bharat's language resources. `Core` handles 25 languages in native script only. `Flex` adds two more languages and three output modes (native script, "mixed" with digits/English inline, and romanized). Both run comfortably on a single modern GPU, or even a Mac with enough unified memory, for moderate throughput.

The five things to know before you ship

1. It will confidently mislabel Bhojpuri as Hindi — and won't tell you

If your audio includes Bhojpuri speakers, the model's automatic language detection gets it wrong **most of the time**, and — this is the important part — it doesn't hedge. On our test clips, when the model heard Bhojpuri and guessed wrong, it was often

90%+ confident it was Hindi. A system that checks "is the model confident?" before trusting a transcript will wave this straight through, because the model genuinely isn't unsure. It just isn't right.

We found this isn't a fixed law of the architecture — Bodhan AI's own `Flex` model handles the same Bhojpuri clips noticeably better (getting the language right about 4 times out of 10, versus 0 out of 10 for `Core`), which tells us this is a data/training artifact that's fixable, not a hard ceiling. But today, right now, if you don't tell the model what language you expect, Bhojpuri content will frequently come back transcribed as if it were Hindi.

What to do: if you know the language ahead of time (a language picker in your app, a caller's known region, a routing rule), pass it explicitly — don't rely on auto-detect for Hindi-belt content (Bhojpuri, Maithili, and to a lesser extent Urdu). If you can't know it ahead of time, budget for a human-review step on anything auto-detected as Hindi from a region where Bhojpuri is common — that's exactly the population getting silently misrouted.

2. Don't use the "smart formatting" mode for money, IDs, or anything that needs to be exactly right

`Flex`'s "mixed mode" is a genuinely appealing feature: instead of writing "पांच हज़ार रुपये," it gives you "₹5,000" — numbers as digits, English words in Latin script, ready to drop into a form or database. It's the kind of feature that looks great in a demo.

In our testing, this mode got the digit *count* right most of the time but the actual value wrong often enough to be dangerous: it turned a spoken "₹19,500" into "₹1,950" — a tenfold error, silently, with no warning. In our sample of currency-bearing clips, mixed mode got the amount exactly right **zero times**; the plain "just write what was said" mode got it right every time. To be fair to the vendor, their own documentation does warn "mixed-script mode will occasionally get it wrong, use native mode for production" — this finding says "occasionally" is doing a lot of work in that sentence.

What to do: for anything where the exact value matters — money, phone numbers, account numbers, dates, quantities — use native mode (the plain, unformatted output) and write your own number/currency extraction on top of it, checked against a validator. Reserve mixed mode for display text where a wrong digit is a cosmetic problem, not a financial one.

3. Real-world noise hurts roughly 10 times more than the model's own benchmark noise did

Every ASR vendor tests "noise robustness" by mixing clean recordings with background noise in software — cafe ambiance, engine hum, a few decibels of static. We did the same thing first, and the model barely blinked: a 2-point-or-less accuracy hit going from clean to synthetically-noisy audio.

Then we tested it on actual recordings of real people talking in real conditions (spontaneous conversation, phone-quality mics, genuine background noise, from a public Indian-languages speech dataset) instead of studio audio with noise mixed in afterward. The accuracy hit was **more than ten times larger** — a real, statistically solid ~20-point drop, not the ~2-point drop the synthetic test predicted. Read: whatever "handles noisy real-world conditions" claim you've seen (from this vendor or frankly most ASR vendors) is very likely benchmarked the easy way, and won't tell you what will happen with your actual users' actual phones in actual crowded rooms.

What to do: before committing to a noise-robustness number from any vendor's spec sheet, run a small pilot on your own real audio — actual customer calls, actual field recordings, not synthetically degraded clean audio. Budget for materially worse accuracy than the marketing numbers imply in noisy environments (markets, streets, moving vehicles, crowded rooms), and plan a fallback (confirmation prompts, human review, push-to-talk in quiet moments) for those conditions rather than assuming the model will cope.

4. If you have multi-speaker audio, a cheap speaker-splitter can make things *worse* than doing nothing

This one is counter-intuitive enough that we want to flag it clearly. The standard recipe for transcribing a conversation with this model is: first figure out who's talking when (diarization), then transcribe each person's turn separately. Sounds sensible.

We tested this on real two-person conversations. Splitting into perfect, human-verified speaker turns *did* help — a real, if modest, improvement over just transcribing the whole recording at once. But a cheap, automatic speaker-splitter (the kind you'd wire up quickly without much tuning) performed **worse than not splitting at all**. Bad speaker boundaries — cutting a sentence in half, merging two speakers into one

segment — hurt the transcription more than the multi-speaker confusion it was meant to fix.

What to do: don't bolt on a basic/naive diarization step and assume it helps by default — test it against the "just transcribe the whole clip" baseline on your own data before shipping it. If you need real multi-speaker support, invest in a proper diarization model (there are good open and commercial options) rather than a quick one; a mediocre one is worse than none.

5. The model's own confidence score is not a reliable safety net

We tried to build a simple "is this transcript trustworthy?" check out of signals the model already gives you — how confident it was about the language, audio quality, clip length, and similar cheap signals. It didn't work: the resulting predictor was barely better than a coin flip at telling good transcripts from bad ones. This connects directly to point #1 above — the model can be very confident and very wrong at the same time, which is exactly the failure mode a naive confidence check is built to catch and, in our tests, didn't.

What to do: don't ship a "trust this transcript if confidence > X" threshold as your only safety mechanism — build in domain-specific checks instead (does the transcript contain the expected entities/format for your use case? does it round-trip sensibly through your downstream logic?), or budget for human review on a random sample regardless of reported confidence.

The numbers, if you want them

Question	Answer	Confidence
Overall word accuracy on clean audio (this test suite)	~78–86%, varies by language	Solid — measured across 8 languages, multiple conditions
Bhojpuri auto-detected correctly	0/10 clips (Core), 4/10 clips (Flex)	Solid for this sample; small sample size
Currency amounts correct in "mixed" formatting mode	0% (vs. 100% in plain mode)	Solid, but based on a small number of currency-bearing clips

Question	Answer	Confidence
Accuracy drop from real-world noise vs. lab-style synthetic noise	~10x larger for real noise	Statistically solid
Multi-speaker accuracy: no split → perfect split → cheap auto-split	Best with perfect split, **worst with cheap auto-split**	Solid direction; small sample (4 recordings)
"Confidence-based" error detection accuracy	Essentially no better than random guessing	Solid negative result

Every number above has a reproducible script and a documented sample size behind it in the full technical report — this table is a summary, not the evidence.

What we haven't tested (be careful here)

- **Code-mixed speech** (Hindi-English "Hinglish" and similar) — untested by us. This is likely common in real Indian-market audio and we have no data on how the model handles it.
- **Languages beyond the 8 we tested** — the model supports 25–27 languages total; we only measured 8. Don't assume our findings transfer to languages we didn't test.
- **Long-form audio** (over ~45 seconds) — the model has a hard length cap and a separate chunking mode for longer audio, which we have not independently verified.
- **Scale/latency/cost at production volume** — this guide is about accuracy and reliability, not throughput or infrastructure cost.

Bottom line

This is a strong, usable model for Indic-language transcription, and the vendor's own documentation is more honest about its limits than most — several of the risks above are things they flag too, just without a number attached. The failure modes here aren't reasons to avoid the model; they're reasons to not take the spec sheet at face value. Test on your own audio, in your own conditions, with your own edge cases —

especially anything Hindi-belt, anything involving money or IDs, and anything with real background noise or multiple speakers — before you trust it in front of a customer.

Full technical report, reproducible test scripts, and every underlying number: see `REPORT.md` *and* `code/` *in this repository.*